Vehicle Detection & Number Plate Recognition Using Image Processing & Deep Learning

Kashish Jain

Course: CISC 695-51-B-2025/Late Summer – Research Methodology & Writing

Project Synopsis

This project implements an end-to-end, educational prototype that detects vehicles in images/videos, localizes license plates, and reads plate text via OCR. It integrates classical computer vision (OpenCV, Haar cascade) and deep learning baselines (CNN, with planned YOLO variants) behind a Flask REST API and an Angular front-end, producing annotated outputs and downloadable results for analysis. The focus is on modularity, clear APIs, and reproducible experiments rather than production surveillance use.

For Project development only

Integrated Development Environment (IDE)

- Backend: VS Code (but any Python IDE works), Flask, Flask-RESTx with Swagger UI at port 5000
- Frontend: Angular dev server at port 4200

Platform

- Development: Windows 10/11

Language

- Python 3.9+ (backend, inference)
- TypeScript / Angular (frontend)

OS with version number

- Windows 10/11

Compiler information

- CPython 3.9+ runtime; Node.js toolchain for Angular build (standard LTS).

Any libraries used (e.g., open-source code)

- OpenCV (CV pipeline), PyTorch/TensorFlow (DL models), Tesseract OCR, Flask/Flask-RESTX, SQLAlchemy, Angular/Bootstrap; COCO 2017 dataset for testing/eval.
- Code references: GitHub repo - https://github.com/jkashish33/Vehicle-Detection-Using-Image-Processing

Brief Build/Installation Instructions

Backend (Flask, Python)

- Create and activate a Python 3.9+ virtual environment (venv)
- pip install -r backend/requirements.txt
- Run python backend/app.py (exposes Swagger at http://localhost:5000/)

Frontend (Angular)

- cd frontend
- npm install

- ng serve (UI at http://localhost:4200/)

Usage:
- Image: POST /vehicle/detect with {file, model}
- Download via /vehicle/download/<processed_filename>
- Video: POST /vehicle/detect_video
- From UI: upload -> choose model (Haar/CNN) -> Submit -> Download

Version Number of Software
Python 3.9+, Flask 3.1+, Angular 20+

Product Inventory
- Backend core: app.py (REST endpoints), detection service modules (e.g., vehicle.py), processed outputs in backend/processed/
- Frontend: Angular components/services (e.g., options-page.component.*, vehicle.service.ts) for upload/config/download flows

Known Issues if any
- Prototype is CPU-bound; real-time throughput is limited without GPU acceleration.
- OCR accuracy degrades on low-quality plates/lighting; preprocessing helps but is not guaranteed.
- Single-user demo; not production-hardened (auth/roles planned for admin endpoints).
- Live streaming stability can vary with RTSP connectivity; retries implemented.

What was the test environment like? I would want to know all that information, complete with version numbers, so I could recreate what you did as closely as possible.
Hardware:
- CPU: >= 4 logical cores
- CPU is fine for Haar/CNN baselines, but will need a GPU for YOLO experiments
- Disk: > 10GB to store the dataset and the processed dataset

Operating System:
- Windows 10/11 or Ubuntu 20+

Runtime & Versions:
- Python: 3.9
- Node.js: Angular 20+
- Browser: Chrome (current version)

Required libraries/frameworks:
- OpenCV, PyTorch/TensorFlow, Tesseract OCR (ML related)
- Backend: Flask + Flask-RESTx
- DB: SQLite in-memory accessed via SQLAlchemy
- Frontend: Angular + Bootstrap

Networking / Ports:
- Backend: http://localhost:5000/
- Frontend: http://localhost:4200/

Dataset:
- Evaluation dataset: COCO 2017 subset for testing

Setup Steps:
- **Backend**:
  - python -m venv .venv && source .venv/bin/activate
  - pip install -r backend/requirements.txt
  - python backend/app.py # serves Swagger on: 5000
- **Frontend**:
  - cd frontend
  - npm install
  - ng serve